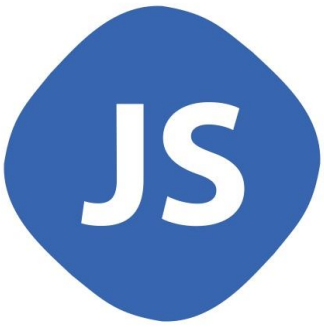


Ведение в DOM

- Введение
- DOM — объектная модель документа
- Критический путь рендеринга (CRP)
- Автоисправление
- Возможности, которые дает DOM

Введение

DOM (**Document Object Model**) была придумана создателями браузеров, и является основным инструментом работы и динамических изменений на странице.

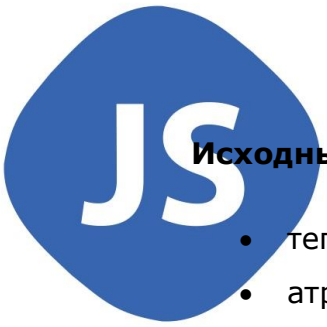
Консорциум World Wide Web Consortium (W3C) стандартизировал DOM как единую спецификацию. Концепция DOM реализована с **разной степенью поддержки** во всех основных браузерах. С помощью **объектной модели** можно **создавать** документы, **перемещаться** по структуре документа, а также **добавлять**, **изменять** или **удалять** элементы и содержимое.

> ***Важно***. DOM является основным инструментом построения и динамических изменений страницы HTML-документа.

Согласно DOM-модели, документ является **иерархией**. Каждый HTML-тег образует отдельный элемент-узел, каждый блок текста - текстовый узел, и т.п.

Другими словами, DOM - это представление документа в виде дерева тегов. Это дерево образуется за счет вложенной структуры тегов плюс текстовые блоки страницы, каждый из которых образует отдельный узел.

Перед тем, как перейти к изучению объектной модели документа необходимо вспомнить, каким образом формируется исходный код веб-страницы (HTML-документа).



Исходный код веб-страницы состоит:

- теги;
- атрибуты;
- текст;
- комментарии.

Теги — это базовая синтаксическая конструкция HTML. Большинство из них являются **парными**. В этом случае один из них является **открывающим**, а другой – **закрывающим**. Одна такая пара тегов образует **HTML-элемент**. HTML-элементы могут иметь дополнительные параметры – **атрибуты**.

В документе для создания определённой разметки одни элементы находятся внутри других. В результате HTML-документ можно представить как множество вложенных друг в друга HTML-элементов.

В качестве примера рассмотрим следующий HTML код (обратите внимание на комментарий в конце исходного документа).

example_1. HTML-код простого документа

```
<!DOCTYPE HTML>
<html lang="ru">
	<head>
		<meta charset="utf-8">
		<title>Программирование в среде JavaScript</title>
		<link href="style.css" rel="stylesheet">
	</head>
	<body>
		<h1>Веб-программирование и дизайн</h1>
		<p>pechora-PROger это:</p>
		<ul>
			<li>все о веб-технологиях</li>
			<li>примеры кодов</li>
```



```
        </ul>
        <!-- и многое другое -->
    </body>
</html>
```

В этом коде корневым элементом является элемент **html**.

- элемент **html** содержит вложенные элементы **head** и **body**;
- элемент **head** содержит вложенные **meta**, **title**, **link**;
- элемент **body** содержит вложенные **h1**, **p**, **ul, комментарий**;
- элемент **ul** в свою очередь, содержит вложенные элементы **li**.

Теперь рассмотрим, как браузер на основании HTML-кода строит DOM-дерево.

DOM — объектная модель документа

Когда пользователь, запрашивает определенную веб-страницу, то в ответе от сервера получает её исходный HTML-код. Взаимодействие пользователя с сервером происходит с помощью клиентского программного обеспечения, браузера.

Браузеру предстоит сделать много работы, прежде чем HTML-ответ сервера будет преобразован в картинку на экране пользователя. Последовательность этих шагов, необходимая для первого отображения страницы, называется **критический путь рендеринга** (Critical Rendering Path, **CRP**).

Знание о **CRP** невероятно полезно для понимания того, как улучшить производительность сайта.

Существует шесть этапов CRP:

1. **Построение DOM-дерева.**
2. **Построение CSSOM-дерева.**
3. **Исполнение JavaScript.**
4. **Построение Render-дерева (дерево визуализации).**
5. **Генерация раскладки (компоновка).**

6. **Отрисовка.**

Примерный алгоритм построения страницы браузером можно описать следующим образом.

- Механизм рендеринга браузера **разделяет** HTML, CSS и JS-файлы полученного кода запрошенной им веб-страницы.
- После разделения содержимого страницы, **HTML** и **CSS**-файлы **анализируются по частям**. В процессе анализа и разбора входных HTML и CSS данных браузер строит на их основе **DOM** и **CSSOM**-деревья (**CSS Object Model**). Деревья являются независимыми объектами, которые отражают различные аспекты документа: один описывает **содержимое**, а другой описывает **правила стиля**, которые необходимо применить к документу.
- **Дерево визуализации** создается путем **объединения** деревьев **DOM** и **CSSOM**, и содержит как информацию о порядке элементов, так и информацию об их стиле. На этом этапе каждому узлу присваиваются специальные координаты, чтобы гарантировать, что каждый элемент будет правильно позиционироваться на экране.
- После выполнения этого действия браузер приступает к **отображению страницы** путем обхода дерева визуализации, и каждый узел, упомянутый в дереве, отображается на экране. В этом процессе браузер использует созданное таким образом **DOM-дерево, а не исходный HTML-код**.

> ***Важно***. DOM – это объектная модель документа, которую браузер создаёт в памяти компьютера на основании HTML-кода, полученного им от сервера.

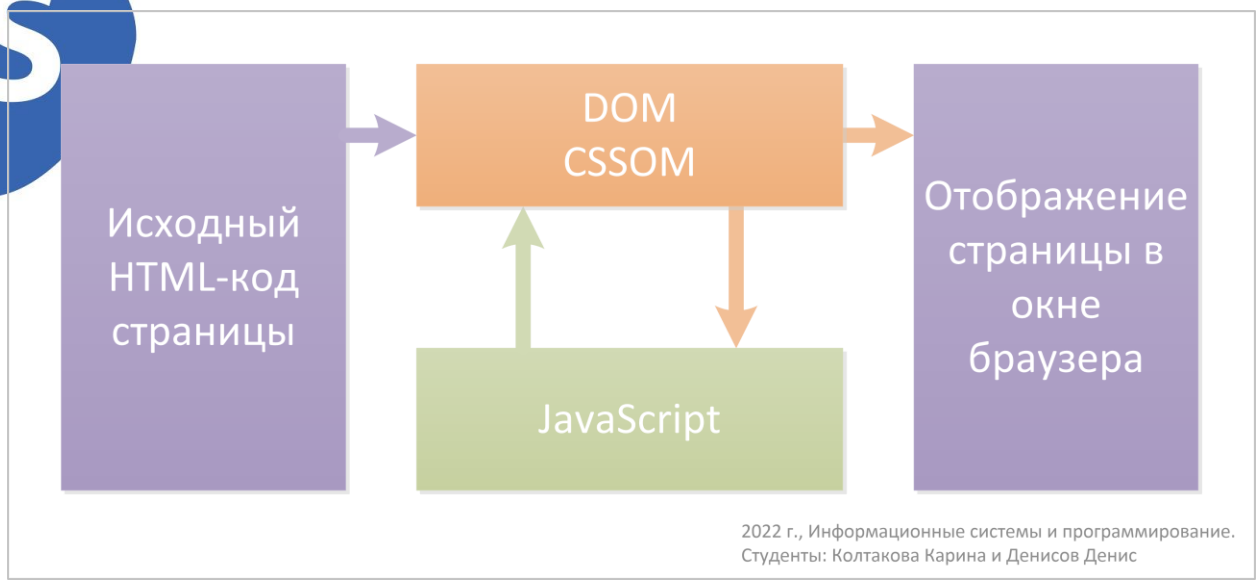


Рис. 1. Построение страницы браузером

Другими словами, HTML-код – это исходный текст страницы, а DOM – это набор связанных объектов, созданных браузером при синтаксическом анализе (парсинге) её текста.

Критический путь рендеринга (CRP)

Загрузка веб-страницы или приложения начинается с запроса HTML. Сервер возвращает HTTP-ответ, состоящий из **заголовков** и **тела** ответа. Именно в теле ответа содержится HTML-документ.

Браузер начинает парсить загружаемый HTML, преобразуя полученные байты документа в DOM-дерево. Браузер создаёт новый запрос каждый раз, когда он находит **ссылки на внешние ресурсы**, будь то файлы стилей, скриптов или ссылки на изображения.

Некоторые запросы являются **блокирующими**. Это означает, что пока такие запросы выполняются - другие запросы приостанавливаются.

Браузер продолжает парсить HTML и создавать DOM до тех пор, пока процесс получения исходного кода HTML не подходит к концу. После завершения построения DOM, браузер конструирует CSS модель. Как только эти модели



сформированы, браузер строит дерево рендера (визуализации), в котором **вычисляет стили для каждого видимого элемента страницы**.

После формирования дерева визуализации происходит компоновка, которая определяет положение и размеры элементов этого дерева. Как только этап завершён - страница рендерится, или еще говорят – **отрисовывается** на экране.

Дерево DOM

Объектная модель документов (DOM) является основным инструментом динамических изменений на странице. DOM дерево это объект, представляющий полностью разобранную HTML-страницу.

Содержимое HTML-страницы превращается в узлы. Начиная с корневого элемента страницы узлы образуются практически из всего, что есть в исходном документе HTML (тегов, текстового контента, комментариев, переносов строк и т.д.).

Элементы, вложенные в другие элементы, представлены в виде дочерних узлов образуя таким образом иерархию узлов. Каждый узел содержит всю необходимую информацию об HTML-элементе, соответствующем этому узлу. Чем больше количество узлов имеет приложение, тем дольше происходит формирование дерева DOM, а значит дольше происходит обработка критических этапов рендеринга.

Связи между узлами в модели формируются на основании того, как HTML-элементы **расположены в коде относительно друг друга**.

HTML-код может быть исполнен по частям. Документ не должен быть загружен полностью для того, чтобы контент начал появляться на странице. Однако, другие ресурсы, такие как CSS и JavaScript, могут блокировать отрисовку страницы.

При этом DOM документа, после его формирования, можно изменять. При изменении DOM браузер практически мгновенно **перерисовывает**

изображение страницы. В результате отображение страницы всегда соответствует DOM.

Для чтения и изменения DOM программно, браузер предоставляет нам DOM API или, другими словами, программный интерфейс.

DOM — это API для HTML документов. API расшифровывается как **интерфейс прикладного программирования**, который можно определить как совокупность способов связи между различными компонентами программного обеспечения.

> ***Другими словами***. DOM API – это набор огромного количества различных объектов, их свойств и методов, которые мы можем использовать для чтения и изменения DOM.

DOM-представление дерева HTML-документа, представленного в example_2, приводится на рисунке 2.

example_2. HTML-документ для построения дерева визуализации

```
<!DOCTYPE HTML>
<html lang="ru">
      <head>
            <meta charset="utf-8">
            <title>Программирование в среде JavaScript</title>
            <link href="style.css" rel="stylesheet">
      </head>
      <body>
            <h1>Веб-программирование и дизайн</h1>
            <p>pechora-PROger это:</p>
            <ul>
                  <li>все о веб-технологиях</li>
                  <li>примеры кодов</li>
```

```
            <li>работы студентов</li>
            <li>обширная практика применения</li>
        </ul>
        <!-- и многое другое -->
    </body>
</html>
```

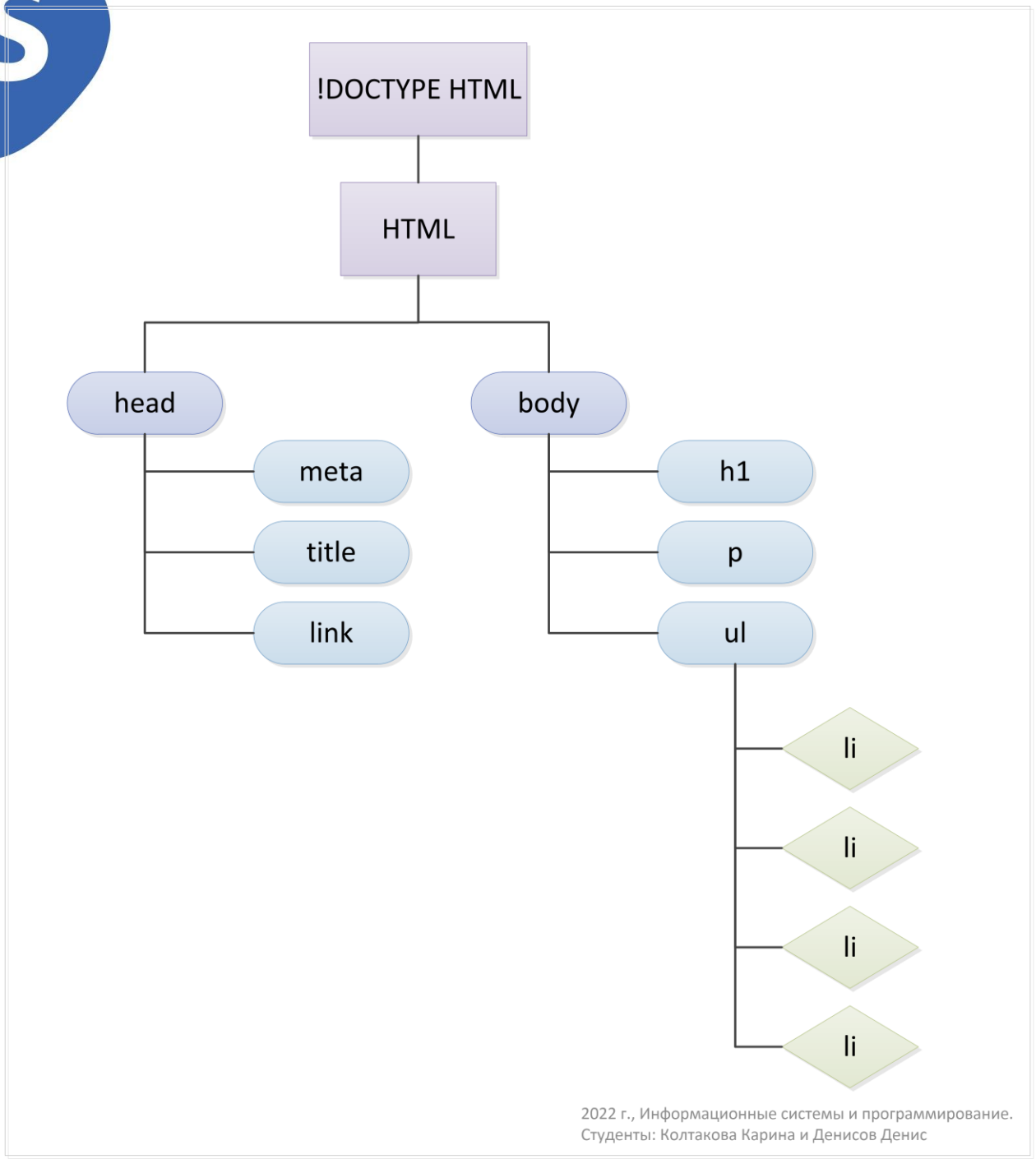


Рис. 2. Дерево DOM

К сведению. DOM — это представление документа в виде дерева тегов, доступное для изменения через JavaScript.

Дерево CSSOM

DOM несёт в себе всё содержимое страницы. CSSOM содержит все стили страницы, то есть данные о том, как стилизовать DOM.

CSSOM (объектная модель CSS) — это объект, представляющий **стили**, связанные с DOM. Он выглядит так же, как DOM, но с соответствующими стилями для каждого узла. Не имеет значения были ли стили объявлены явно или наследуются.

CSSOM-представление файла style.css, представленного в example_3, приводится на рисунке 3.

example_3. Файл style.css

```
body {
      background: #f5f5f5;
}
h1 {
      text-align:center;
      font-size:2em;
}
p{
      margin:10px 0;
      padding:20px;
}
ul{
      list-style-type:square;
}
```

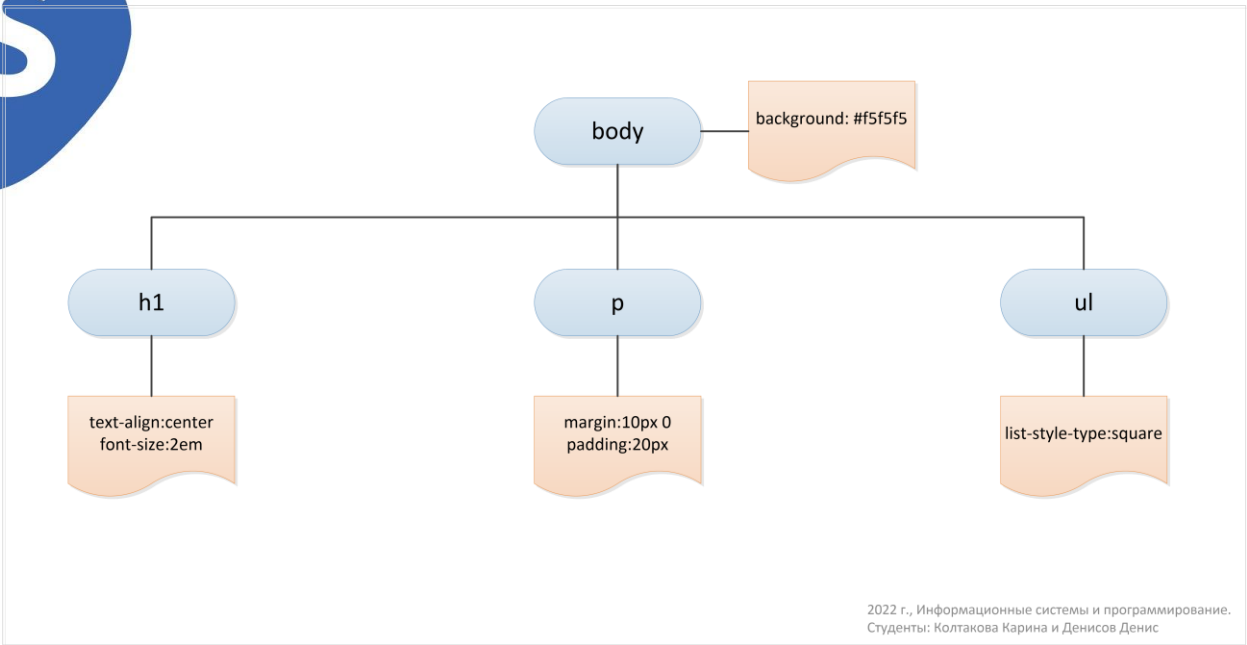


Рис. 3. Дерево CSSOM

CSS считается **блокирующим обработку ресурсом**. Это значит, что рендер-дерево (см. ниже) не может быть построено без полного первоначального разбора CSS.

В отличии от HTML, CSS не может быть использован по частям в силу своей каскадной природы. Стили, описанные в документе ниже, могут переопределять и изменять стили, определённые ранее. Так что, если мы начнём использовать CSS-стили до того, как будет разобрана таблица стилей, мы можем столкнуться с ситуацией, когда стили будут применяться неверно. Это означает, что для перехода к следующему шагу, необходимо полностью разобрать CSS.

CSS также может являться **блокирующим скрипты**, потому что JavaScript-файлы должны дождаться построения CSSOM, прежде чем начать исполняться.

Исполнение JavaScript

Для работы с DOM в большинстве случаев используется JavaScript, т.к. на сегодняшний день это самый популярный язык программирования скриптов, исполняемых браузером.



JavaScript является **блокирующим ресурсом для парсера**. Это означает, что JavaScript блокирует разбор самого HTML-документа.

Когда парсер доходит до тега **script** (не важно внутренний он или внешний), он останавливается, забирает файл (если он внешний) и запускает его. Вот почему, если мы имеем JavaScript-файл, который ссылается на элементы документа, мы обязательно должны поместить его после их появления.

JavaScript можно загружать асинхронно, указав атрибут **async** для того, чтобы избежать блокировки парсера.

Дерево визуализации

Согласно DOM-модели, документ является иерархией, деревом. Когда происходит загрузка соответствующей страницы, в памяти браузера реализуется поддержка дерева узлов, сгенерированных согласно **использованным в документе HTML-тегам и их стилям**.

Рендер-дерево (дерево визуализации, дерево узлов) — это совокупность DOM и CSSOM. Это дерево, которое даёт представление о том, что в конечном итоге будет отображено на странице.

HTML-теги являются **узлами-элементами** (или просто **элементами**). Они образуют структуру дерева.

Текст внутри элементов образует **текстовые узлы**, обозначенные как **#text**. Текстовый узел содержит в себе только строку текста. У него не может быть потомков, т.е. он находится всегда на самом нижнем уровне.

При изучении дерева узлов, обратите внимание на специальные символы:

- перевод строки: ↲ (в JavaScript обозначается \n)
- пробел: ␣

Пробелы и **переводы** строки – это полноправные символы, как буквы и цифры. Они образуют **пустые текстовые узлы** и становятся частью дерева DOM.



Если в документе удалить пробелы (или любые другие пробельные символы), то в дереве DOM их тоже не будет. В фрагменте HTML-кода, приведенного ниже, пробельных текстовых узлов нет:

```
<!DOCTYPE HTML>
<html lang="ru"><head><meta charset="utf-8"><title>Программирование в среде JavaScript</title><link href="style.css" rel="stylesheet"></head>...
```

> ***Важно***. В примере example_2 в теге **head** есть несколько пробелов перед тегом **meta**, которые образуют **текстовый узел #text**, он содержит в себе только перенос строки и несколько пробелов.

Все, что есть в документе, даже **комментарии**, является частью дерева DOM.

Директива **DOCTYPE**, которую мы ставим в начале документа HTML, тоже является DOM-узлом.

> ***Важно***. **Узел** — фундаментальный строительный блок модели. Узел — это общее имя для объекта любого типа в иерархии DOM.

Каждый узел дерева DOM – это **объект**. По той причине, что даже небольшой HTML-код будет выглядеть громоздким в представлении дерева узлов DOM, схематически отобразить дерево - задача непростая.

Используя деревья DOM и CSSOM, представленные выше, построим дерево визуализации (рендер-дерево) узлов.

С некоторой долей условности полученное дерево представлено на рисунке 4.

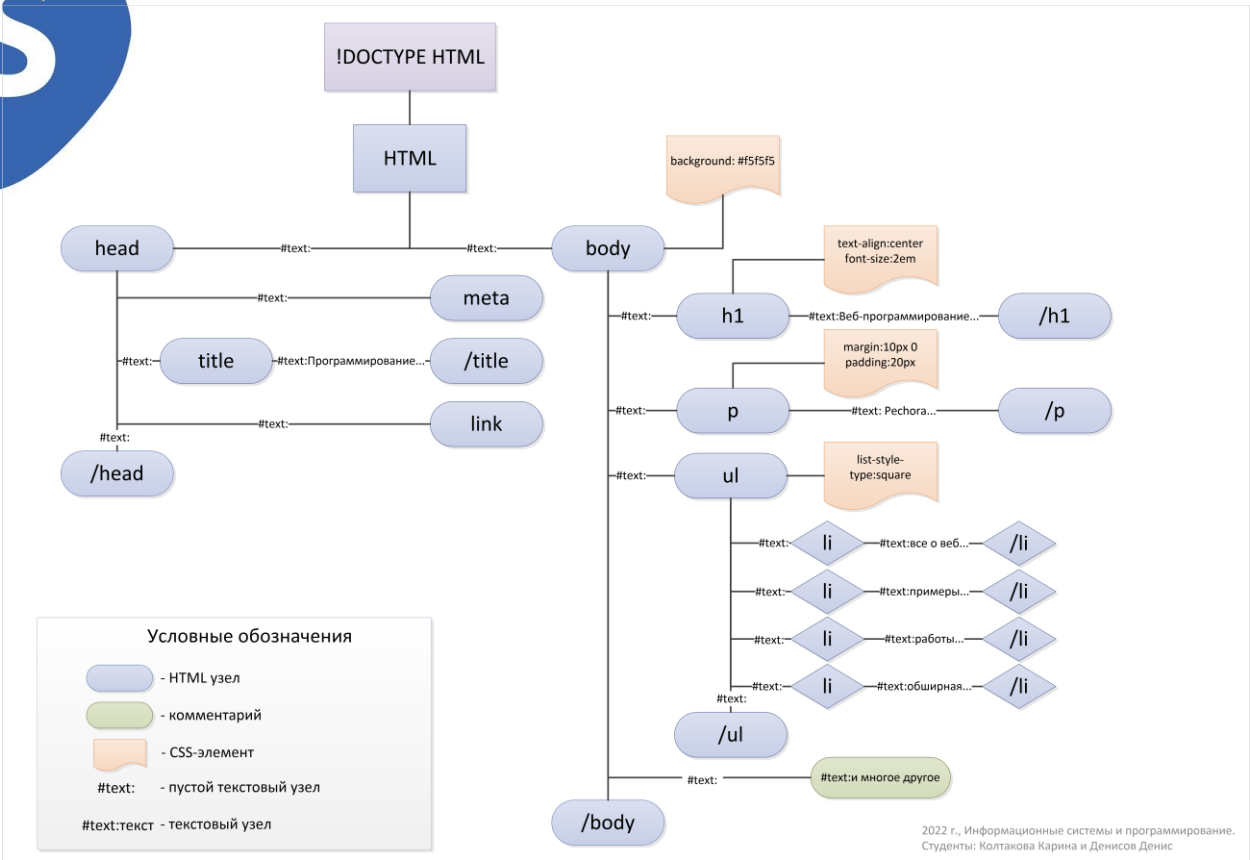


Рис. 4. Дерево узлов DOM.

Существует 12 типов узлов. Но на практике мы в основном работаем с 4 из них:

- **document** – "входная точка" в DOM;
- **узлы-элементы** – HTML-теги (основные строительные блоки);
- **текстовые узлы** – содержат текст (в том числе пустой текст);
- **комментарии** – иногда в них можно включить информацию, которая не будет показана, но доступна в DOM для чтения с помощью JS.

> ***Важно***. Дерево состоит из **узлов**, но только некоторые из них являются **HTML-элементами**.

У каждого узла есть родительский узел (за исключением корневого узла) и любое количество дочерних узлов. Одни типы узлов могут иметь дочерние узлы различных типов, другие являются так называемыми **листовыми узлами**, у которых не может быть дочерних узлов.



Компоновка

В тот момент, когда дерево рендера построено, становится возможным этап компоновки (раскладки). Компоновка зависит от размеров экрана. Этот этап определяет, где и как на странице будут спозиционированы элементы и каковы связи между элементами.

Компоновка — это то, что определяет размер видимой области документа (viewport), которая обеспечивает контекст для стилей CSS, зависимых от него, например, проценты или единицы вьюпорта.

Мета тег вьюпорт, который может быть указан в теге **head** страницы, определяет ширину видимой области и влияет на компоновку. Без этого тега браузеры используют ширину "по умолчанию", которая обычно составляет 960px.

Например, наиболее частым значением для этого мета тега является размер, соответствующий ширине устройства.

```
<meta name="viewport" content="width=device-width,initial-scale=1">
```

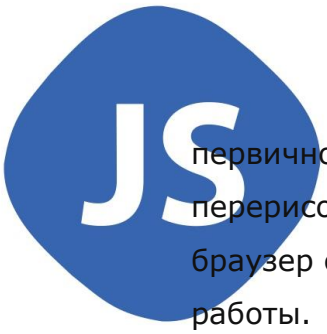
Этот тег установит ширину видимой области в 100% от ширины экрана устройства вместо того, чтобы использовать ширину по умолчанию.

Эта ширина (device-width) изменяется каждый раз, когда пользователь поворачивает телефон. Это приводит к запуску этапа компоновки. Равно как и при изменении размеров окна в обычном браузере.

На производительность компоновки непосредственно влияет DOM - чем больше узлов в вашем документе, тем больше времени понадобится на перерасчёт позиций и размеров всех элементов. Компоновка может стать узким местом, ведущим к зависаниям, особенно если выполняется одновременно со скроллом или другой анимацией.

Отрисовка

Последний этап – отрисовка пикселей на экране. Когда дерево рендера создано, компоновка произошла, пиксели могут быть отрисованы. При



первичной загрузке документа весь экран будет отрисован. После этого будут перерисовываться только необходимые к обновлению части экрана, так как браузер старается оптимизировать процесс отрисовки, избегая ненужной работы.

Время, которое займет этот этап, зависит как от величины DOM, так и от того, какие стили применяются. Некоторые стили требуют больше усилий, чтобы быть применёнными, чем другие. Например, сложное градиентное фоновое изображение потребует больше времени, чем простой сплошной цвет на фоне.

> ***Примечание***. Знание этапов помогает оптимизировать структуру документа и уменьшить время загрузки страницы. Но необходимо помнить, никакая оптимизация не должна ухудшать **юзабилити** вашего сайта.

Автоисправление

Если браузер сталкивается с некорректно написанным HTML-кодом, он **автоматически корректирует** его при построении DOM.

Например, в начале документа всегда должен быть тег **html**. Даже если его нет в документе – он будет в дереве DOM, браузер его создаст. То же самое касается и тега **body**.

Например, если HTML-файл состоит из единственного слова "Привет", браузер обернёт его в теги **html** и **body**, добавит необходимый тег **head**, и документ будет выглядеть так:

```
<!DOCTYPE HTML>
<html>
	<head></head>
	<body>
		Привет
	</body>
```

```
</html>
```

При генерации DOM браузер самостоятельно обрабатывает ошибки в документе, закрывает теги и так далее. Поэтому, если есть документ с незакрытыми тегами:

example_4. Автоисправление

```
<!DOCTYPE HTML>
<html lang="ru">
      <head>
            <meta charset="utf-8">
            <title>Программирование в среде JavaScript</title>
            <link href="style.css" rel="stylesheet">
      </head>
      <body>
            <h1>Веб-программирование и дизайн</h1>
            <p>Изучаем веб-технологии
            <li>php
            <li>javascript
            <li>html
      </body>
</html>
```

… то DOM будет нормальным, потому что браузер сам закроет теги и восстановит отсутствующие детали:

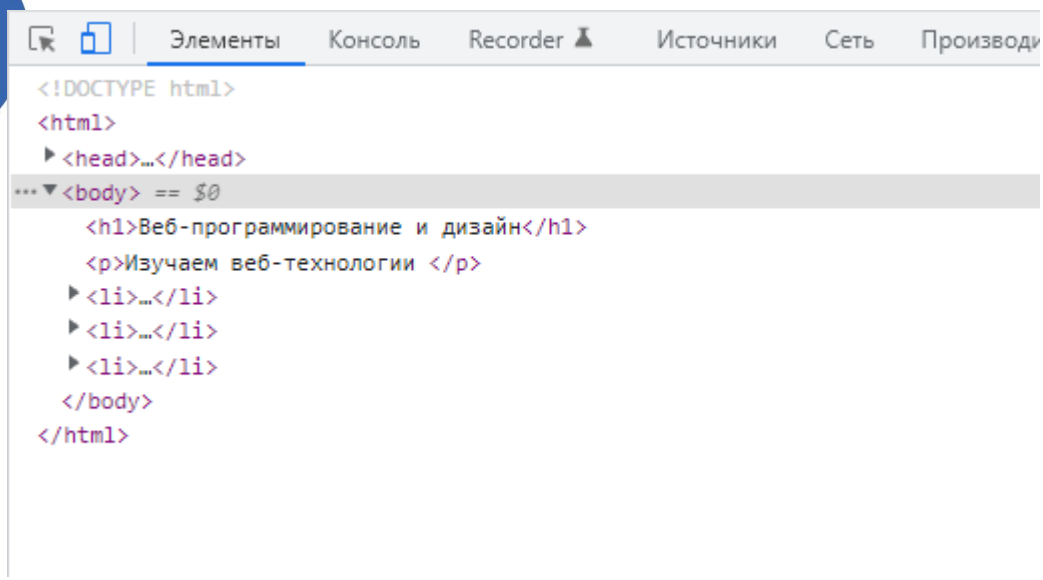


Рис. 5. Автоисправление

Возможности, которые дает DOM

DOM API нужен, для реализации возможности, используя JavaScript изменять страницу на "лету", то есть, делать её **динамической** и **интерактивной**.

DOM API предоставляет разработчикам огромное количество методов, с помощью которых он может создавать сложные интерфейсы, формы, выполнять обработку действий пользователей, добавлять и удалять различные элементы на странице, изменять их содержимое, свойства (атрибуты), и многое другое.

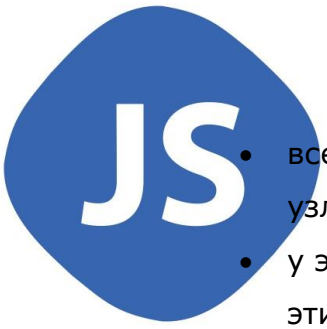
Сейчас в вебе практически нет сайтов, в сценариях которых отсутствовала бы работа с DOM.

> ***Важно***. DOM нужен для того, чтобы манипулировать страницей — читать информацию из HTML, создавать и изменять элементы.

Фактически, DOM предоставляет возможность делать со страницей всё, что угодно.

Итого

- DOM-модель — это внутреннее представление HTML-страницы в виде дерева;



- все элементы страницы, включая теги, текст, комментарии, являются узлами DOM;
- у элементов DOM есть методы, которые позволяют взаимодействовать с этими элементами;
- DOM-модель используется не только в JavaScript, это известный способ представления XML-документов.

Выполняем **самостоятельные работы**, закрепляем материал:

- Используя интернет-ресурс строим DOM предложенного документа.
- Создаем код HTML-страницы по существующему дереву DOM документа.
- Создаем код HTML-страницы по существующим деревьям DOM и CSSOM.
- Выполняем отладку JS-сценария.

